Experiment No.: 01
Title: Interpretation and Identification of test cases for given problem statement

Batch: C3 **Roll No: 16014225801** **Experiment No: 01**

Aim: Interpretation and Identification of test cases for given problem statement

---

Resources needed: Text Editor, C/C++ IDE

Theory:

In competitive programming the problem statement is mostly given pertaining to real-world scenario. Along with input and output information, constraints on input/output are also given most of the times with the problem statement. Hence in competitive programming to get started with a problem, the first step is to read and understand the problem statement and given information and find the following details from it:

1. Identify the input values
2. Identify the constraints on input
3. Identify the output values
4. Identify the constraints on output

Along with problem statement and constraints information, input format and output format information is also given. To get a clear understanding of these formats, sample input values and its corresponding output values are also given. We refer to these values as sample test cases.

A Test Case is some sample input value and it's expected out value. In competitive programming, with every problem statement, some sample test cases are given most of the times. But these sample test cases do not cover all the general and special cases. Since the competitive programming platforms evaluate the solution based on test cases, it is essential to identify the general and special test cases for the problem statement under consideration. Special Cases mostly require special handling in the solution.

Test Cases can be identified using following approach:

1. Random Value Test Cases – Here some random values of input and it's corresponding output within the given input/output constraints can be considered
2. Minimal Value Test Cases – Here considering factors, such as minimum number of total input values or minimum possible value for a input, special cases can be identified
3. Maximal Value Test Cases - Here considering factors, such as maximum number of total input values or maximum possible value for a input, special cases can be identified. These test cases also help to determine problems like integer overflow, crash point of solution, maximum running time and memory requirement of the solution and so on.

Procedure

Consider the following problem statement and other information provided along with it:

Problem: You are provided an array of size that contains non-negative integers. Your task is to determine whether the number that is formed by selecting the last digit of all the N numbers is divisible by 10.

Input format

- First line: A single integer N denoting the size of array A

- Second line: N space-separated integers.

Output format

If the number is divisible by 10, then print '*Yes'*. Otherwise, print '*No'*

Constraints

$1 \leq N \leq 105$

$0 \leq A[i] \leq 105$

Sample Input	Sample Output
5 45 23 65 22 74	No

Task 1:

Identify the following from the given information:

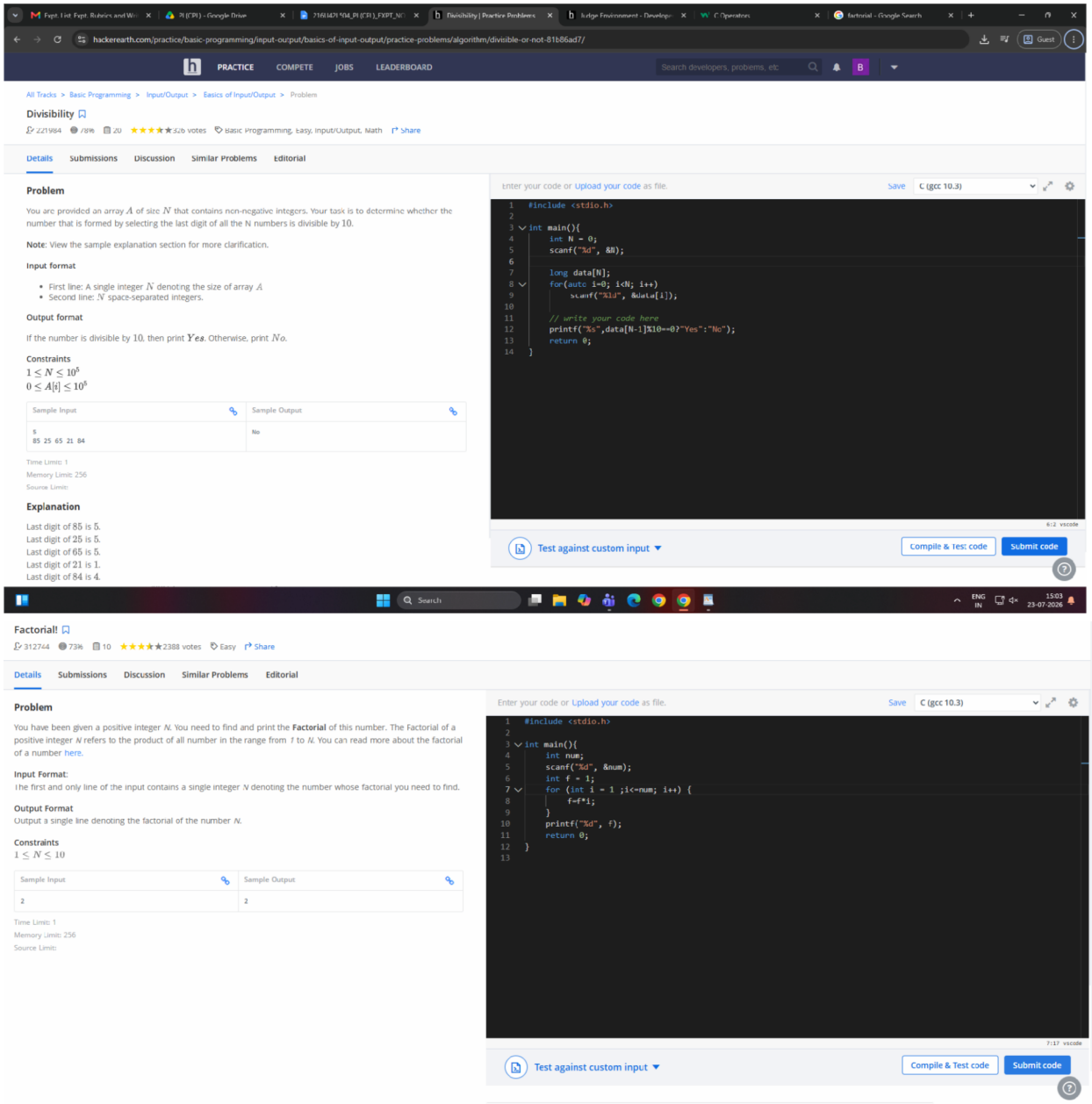
1. Input values
2. Constraints on input values
3. Output values
4. Constraints on output values
5. Specified format for input values
6. Specified format for output values

Task 2:

Identify general and special test cases for given problem statement. List down in all 10 - 12 test cases in table format as shown:

Sr. No.	Sample Input	Sample Output	Description	Test Case Type (general/special)
1.	5 45 23 65 22 74	No	array with 5 integer numbers	general

Results: (Steps with screenshots)



Post Lab Questions:

Consider the given problem statement and related information:

Problem

You have been given a positive integer N where $1 \leq N \leq 12$. You need to find and print the Factorial of this number

Input Format

The first and only line of the input contains a single integer N denoting the number whose factorial you need to find.

Output Format

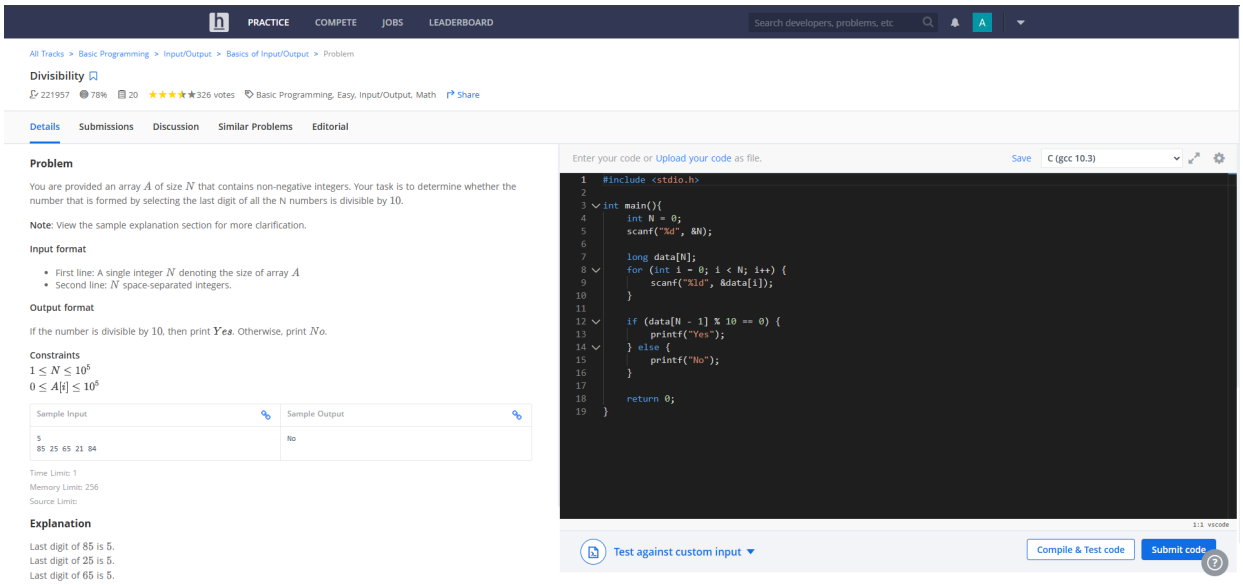
Output a single line denoting the factorial of the number N.

Sample Input	Sample Output
3	6

Task 1:

Identify the following from the given information:

1. Input values
2. Constraints on input values
3. Output values
4. Constraints on output values
5. Specified format for input values
6. Specified format for output values



Task 2:

Identify general and special test cases for given problem statement. List down in all 6-8 test cases in table format (refer activity section for table format of test cases)

Submission ID: 130384374

RESULT: Accepted — Refer judge environment

Score	Time (sec)	Memory (KiB)	Language
20	0.03655	2	C

Input	Result	Time (sec)	Memory (KiB)	Score	Your output	Correct output	Diff
Input #1	Accepted	0.009224	2	25			
Input #2	Accepted	0.008755	2	25			
Input #3	Accepted	0.009848	2	25			
Input #4	Accepted	0.008727	2	25			

Conclusion:

This experiment successfully demonstrated how to interpret problem statements, identify constraints, and construct minimal, maximal, and edge-case test cases. By analyzing these parameters, we derived an optimized logic—checking only the final digit of A[N-1] ensuring maximum efficiency and complete test-case coverage in competitive programming.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

1. Antti Laaksonen, “Guide to Competitive Programming”,Springer,2018
2. Gayle Laakmann McDowell,” Cracking the Coding Interview”,CareerCup LLC,2015
3. Steven S. Skiena Miguel A. Revilla,”Programming challenges, The Programming Contest Training Manual”, Springer, 2006
4. Antti Laaksonen, “Competitive Programmer’s Handbook”, Hand book, 2018
5. Steven Halim and Felix Halim, “Competitive Programming 3: The Lower Bounds of Programming Contests”, Handbook for ACM ICPC